Developing a Priority-Based Rule System for Autonomous Agents in the Pacman Environment

Julia Arukakkal

1. Motivation

The primary goal was to develop a real-time autonomous agent for Pacman using the CLIPS rule-based engine. The challenge lay in creating a system capable of dynamic goal arbitration—balancing the immediate need to collect food, the strategic acquisition of Power Pellets, and the survival instinct required to evade lethal ghosts.

2. Background

2.1 Pacman Mechanics and Scoring

Pacman is a grid-based maze game where the objective is to collect all pellets to clear the level. While ghosts are normally lethal, consuming a **Power Pellet** turns them white and vulnerable. In this "frightened" state, they move slower and can be eaten for a point bonus, temporarily removing them from the board.

2.2 Chief Concerns

The development of the agent focused on three main engineering challenges:

- **Speed and Responsiveness:** To avoid "lag-deaths," where the game state changes before the AI can react, the agent required near-instant decision-making. By using a simple distance formula instead of complex path-searching, the system ensures real-time reactions.
- **Goal Prioritization:** The agent must constantly decide whether to hunt food, chase white ghosts, or flee from danger. Success depends on a weighted system that always prioritizes survival.
- **Decisiveness:** In symmetric mazes, an agent can get "stuck" between two equal choices. A primary concern was ensuring the agent always maintains momentum rather than vibrating in place.

3. Methods

The agent operates through a Hybrid Architecture consisting of a Python interface and a CLIPS production system. It utilizes a Utility-Based method where every legal move (North, South, East, West) accumulates a score based on prioritized rules.

3.1 Survival Rule (Lethal Threat Evasion)

The highest priority rule (highest salience). It identifies ghosts in their dangerous state (when the ghost isn't white). If a dangerous ghost is near a potential move, that direction is assigned a massive negative penalty, ensuring evasion overrides all other goals.

3.2 Hunting Rule (Scared Ghost Pursuit)

This activates when a ghost turns white. The agent calculates the distance to these vulnerable ghosts and assigns a high positive multiplier to moves that bring Pacman closer, shifting the agent from a defensive to an offensive posture.

3.3 Consumption Rule (Food Collection)

This handles the primary objective by evaluating the distance to all remaining pellets. It adds a moderate positive score to moves bringing Pacman closer to food.

3.4 Power-Up Rule (Strategic Aggression)

This rule targets Power Pellets, assigning them a higher weight than regular food but lower than a white ghost, allowing Pacman to strategically invert the game state.

3.5 Momentum Rule (Anti-Stuck Logic)

To solve "vibration" issues, this rule applies a negative penalty to any move that is the direct opposite of the agent's previous action, forcing exploration and forward movement.

3.6 Decision Rule (Best-Action Selection)

This final rule compares aggregated scores and selects the direction with the highest total utility.

4. Pertinent Implementation Details

- **Action Filtering:** The Python layer strips the "Stop" command from the move list, ensuring the agent is forced to stay in motion even under high-threat conditions.
- **Fact Refreshing:** To ensure accuracy, the CLIPS environment is "reset" and re-populated with fresh facts every frame, preventing decisions based on stale ghost positions.
- **Coordinate Mapping:** Facts regarding Pacman, food, and ghost locations are asserted as (x, y) coordinates, allowing the logic to calculate distances relative to Pacman's current position.

5. Results

The agent demonstrated high-speed responsiveness and a seamless transition between defensive evasion and offensive hunting. The heuristic approach provided a significant increase in decision speed over search-based alternatives, allowing the agent to dodge ghosts in tight corridors.

However, the agent was unable to successfully clear the entire maze. Post-trial analysis identified that the reliance on simple distance heuristics meant the agent could not "see" past walls, sometimes leading it into dead-ends or causing it to struggle with pellets that required long, non-linear paths to reach. While the agent was highly reactive and fast, these results suggest that clearing the maze would require a hybrid model that combines rule-based speed with limited long-term pathfinding.

6. Instructions for Execution

To run the agent, the system requires a bridge between the game environment and the inference engine. Follow these steps to initialize the simulation:

6.1 Prerequisites

- **Python 3.x:** Ensure the base environment is installed.
- **CLIPS / CLIPSPy:** The decision engine requires the CLIPS library to process the production rules.
- **Flask:** Used to host the local server that facilitates communication between the game client and the AI.

6.2 System Initialization To ensure the agent and game synchronize correctly, it is recommended to use a **split-terminal** setup to monitor the logic flow alongside the game.

1. **Launch the Logic Server:** Make sure you are in the **pacman_project** directory and can see clips_server.py. Open the first terminal and run the command:

   python clips_server.py

This initializes the Flask server and loads the CLIPS rule base. The terminal will confirm the server is active and listening for state updates.

2. **Start the Game Environment:** Make sure you are in the **pacman_project/berkeley_pacman/main** directory. Open a second terminal and run the specific agent command:

   python pacman.py -p ClipsAgent

The *-p ClipsAgent* flag instructs the game to hand over control to your custom CLIPS-driven logic.

3. **Observation:** Once the connection is established, the following will be visible:
    - **Pacman Frame:** A graphical window will open, displaying the maze and the autonomous movement of the agent.
    - **Action Log:** The first terminal will display a real-time log of every action taken. This includes the direction chosen (e.g., *ACTION: WEST*) and the specific utility scores generated by the rules for each frame.

6.3 Troubleshooting

- **Connection Refused:** Always ensure the *clips_server.py* (server) is running before launching *pacman.py* (client).